

MCP Server — Cloud Run Deployment Guide

Day 8, Goal 2: Auth, Rate Limiting, Logging & Production Deployment

1. Project Architecture & Overview

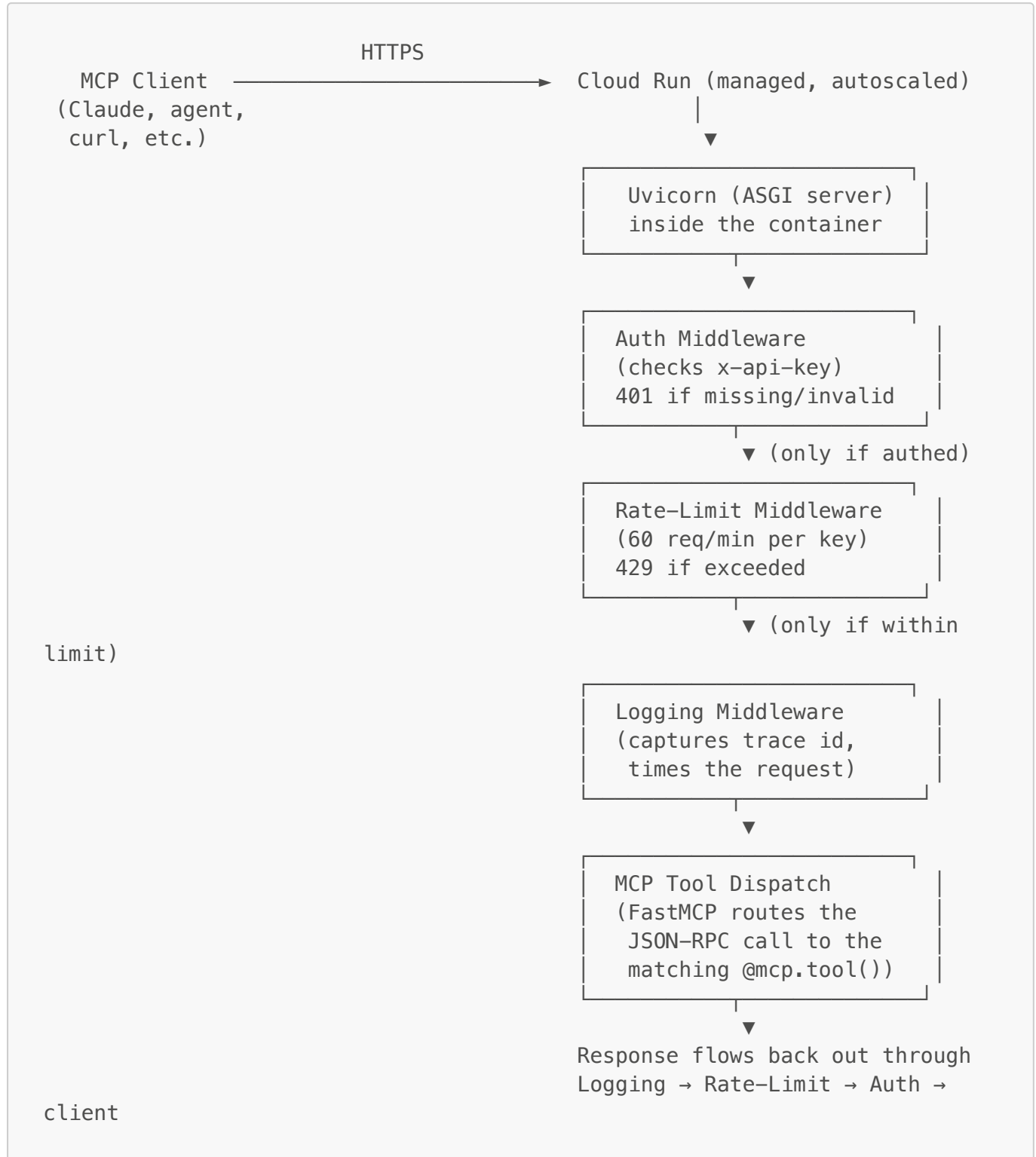
1.1 Stated assumptions (read this first)

This guide is built on a specific set of technical choices that were **assumed, not handed to us as hard requirements** in the original task description. They are reasonable, defensible defaults for this exercise — but they are choices, and you should be able to articulate *why* each one was made and what you'd change in a real production rollout. Call these out explicitly when you present this work:

- **Runtime stack:** Python 3.12, MCP server built on the official `mcp` Python SDK (`mcp.server.fastmcp.FastMCP`), exposed over an ASGI app via FastAPI, using the **streamable-HTTP** transport (not stdio, not SSE) so it's reachable as a normal HTTPS endpoint behind Cloud Run. SSE is legacy at this point; streamable-HTTP is the SDK's recommended transport for new server-side deployments.
- **Error-handling philosophy: strict / fail-fast.**
 - Missing or invalid `x-api-key` → immediate **401**, no fallback, no "let it through and log a warning."
 - Exceeded rate limit → immediate **429** with a `Retry-After` header.
 - Missing required environment variables (`VALID_API_KEYS`) at container startup → the process **exits non-zero immediately**. It does not start up in a degraded "no auth" mode. A server that fails to boot is a much smaller incident than a server that boots with auth silently disabled.
- **Rate limiting is an in-memory sliding window, scoped per container instance.** This is explicitly called out as a **limitation**, not a design we'd defend in a real multi-instance production deployment: if Cloud Run scales this service past 1 instance, each instance keeps its own independent counter, so the *effective* limit becomes $60 \times (\text{number of instances})$ requests/minute per key, not 60. For this exercise we work around it with `gcloud run deploy --min-instances=1 --max-instances=1`, which pins the service to exactly one instance so the single in-memory counter is authoritative. The real fix — and what you'd actually ship to a client — is a shared external store (Redis / Cloud Memorystore) that every instance reads and writes the same counters against, so the limit is correct regardless of how many instances Cloud Run spins up. We note this again in Section 4.
- **API keys live in a `VALID_API_KEYS` environment variable**, formatted as comma-separated `key:client_name` pairs (e.g. `sk_abc123:acme-corp,sk_xyz789:globex-inc`), for this exercise. This is fine for a single intern exercise and a handful of trusted clients. It is **not** how you'd manage keys for a real client base — Section 5's `setup.sh` and the deploy flow assume this env-var model, but `auth_setup.md` (the deliverable you write after deploying) should describe the production-grade alternative: keys minted and stored in **Google Secret Manager**, mounted into Cloud Run as secret env vars or volumes, with rotation handled by issuing a new secret version and updating the Cloud Run service to point at it — never by editing plaintext env vars in place.

- **Placeholders you must fill in before anything in this guide will actually run:** `YOUR_PROJECT_ID` (your real GCP project ID), region (defaults to `us-central1` throughout — change every occurrence if you deploy elsewhere), and the Artifact Registry repo name `mcp-server-repo` (change if you want a different name, but then change it everywhere it appears below too).

1.2 Request lifecycle, end to end



A request that fails auth never reaches the rate limiter, and a request that's throttled never reaches the tool dispatch logic or burns "real" application time — each layer is a hard gate, not a soft check, in keeping with the fail-fast philosophy above.

1.3 Why this stack

- **FastAPI / ASGI as the transport layer:** the official MCP Python SDK's `FastMCP` class can run standalone, but it can also hand you a mountable ASGI app (`.streamable_http_app()`) that drops cleanly into an existing FastAPI application. That gives us FastAPI's middleware stack, dependency injection, and routing for free, while the MCP SDK still owns the actual protocol logic (JSON-RPC framing, tool registration, schema generation from type hints). We don't have to hand-roll any of the MCP wire protocol ourselves.
- **Docker multi-stage build:** a build stage that installs Python dependencies (which need compilers and build tooling for some packages) and a separate, slim runtime stage that copies in only the installed virtual environment plus application code. This keeps the final image small, which matters twice on Cloud Run specifically: smaller images pull faster (a real factor in cold-start latency) and a smaller image has a smaller attack surface — no compilers, no build caches, no apt lists sitting in the layer that actually runs in production.
- **Cloud Run for hosting:** it's a fully managed, autoscaling container platform billed per-request/per-100ms-of-CPU, which fits an MCP server's traffic pattern (bursty, client-driven, idle most of the time) far better than a statically-provisioned VM. We trade away some control (no SSH into the box, no persistent local disk) for not having to manage a fleet at all. The one place this trade-off bites us is the in-memory rate limiter (Section 1.1) — autoscaling and a per-instance in-memory counter are fundamentally in tension, which is why we pin instance count for this exercise.
- **Cloud Logging + Cloud Trace instead of self-hosted observability:** Cloud Run automatically forwards anything a container writes to stdout/stderr into Cloud Logging as structured log entries if the payload is valid JSON, and it automatically injects an `X-Cloud-Trace-Context` header on every inbound request that ties back to a Cloud Trace span. We get both a centralized log store and distributed tracing without running our own ELK stack or Jaeger collector — we just need to (a) log JSON to stdout and (b) read and propagate that header, both of which `middleware/logging.py` does.

2. Repository & Folder Structure

```
mcp_server/
├── server.py
├── requirements.txt
├── Dockerfile
├── deploy.sh
├── .env.example
├── .dockerignore
├── middleware/
│   ├── __init__.py
│   ├── auth.py
│   ├── rate_limit.py
│   └── logging.py
mcp_deploy/
├── cloud_run_url.txt
├── trace_ids.md
├── auth_setup.md
README.md
```

```
questions.md
output.log
```

Scaffold script — macOS/zsh, copy-pasteable, idempotent (safe to run more than once; it never overwrites a file or directory that already exists, it only fills in what's missing):

```
#!/usr/bin/env zsh
set -euo pipefail

# scaffold.sh — creates the mcp_server / mcp_deploy project layout and a
# Python virtual environment named `myenv` (not `venv`), in the current
# directory. Safe to re-run: mkdir -p and touch are no-ops on existing
# paths, and the venv step checks for an existing ./myenv before creating
# one.

ROOT="$(pwd)"

echo "==> Creating directory structure under ${ROOT}"
mkdir -p "${ROOT}/mcp_server/middleware"
mkdir -p "${ROOT}/mcp_deploy"

echo "==> Creating mcp_server/ files"
touch "${ROOT}/mcp_server/server.py"
touch "${ROOT}/mcp_server/requirements.txt"
touch "${ROOT}/mcp_server/Dockerfile"
touch "${ROOT}/mcp_server/deploy.sh"
chmod +x "${ROOT}/mcp_server/deploy.sh"
touch "${ROOT}/mcp_server/.env.example"
touch "${ROOT}/mcp_server/.dockerignore"

echo "==> Creating mcp_server/middleware/ files"
touch "${ROOT}/mcp_server/middleware/__init__.py"
touch "${ROOT}/mcp_server/middleware/auth.py"
touch "${ROOT}/mcp_server/middleware/rate_limit.py"
touch "${ROOT}/mcp_server/middleware/logging.py"

echo "==> Creating mcp_deploy/ files"
touch "${ROOT}/mcp_deploy/cloud_run_url.txt"
touch "${ROOT}/mcp_deploy/trace_ids.md"
touch "${ROOT}/mcp_deploy/auth_setup.md"

echo "==> Creating top-level files"
touch "${ROOT}/README.md"
touch "${ROOT}/questions.md"
touch "${ROOT}/output.log"

echo "==> Setting up Python virtual environment (myenv)"
if [[ ! -d "${ROOT}/myenv" ]]; then
    python3 -m venv myenv
    echo "    created ./myenv"
else
```

```

    echo "    ./myenv already exists, skipping"
fi

echo "==> Done. Structure created/verified under ${ROOT}"
echo "    Activate the venv with: source myenv/bin/activate"

```

3. Production-Ready Implementation Code

3.1 `mcp_server/Dockerfile`

```

# syntax=docker/dockerfile:1

# -----
--
# Stage 1: builder
# Installs Python dependencies into an isolated virtual environment. This
# stage carries pip's build tooling and any wheel-build dependencies, none
# of which need to exist in the final runtime image.
# -----
--
FROM python:3.12-slim AS builder

WORKDIR /build

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:${PATH}"

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# -----
--
# Stage 2: runtime
# Slim final image: only the already-built virtual environment and the
# application source are copied in. No compilers, no pip cache, no build
# tooling – smaller image, smaller attack surface, faster cold pulls.
# -----
--
FROM python:3.12-slim AS runtime

# Non-root user: Cloud Run does not require this, but running as root
# inside the container is an avoidable privilege-escalation risk if the
# application is ever compromised.
RUN groupadd --system mcpgroup && \
    useradd --system --gid mcpgroup --create-home --home-dir /home/mcpuser
mcpuser

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:${PATH}"

```

```

WORKDIR /app
COPY server.py ./server.py
COPY middleware ./middleware

RUN chown -R mcpuser:mcpgroup /app
USER mcpuser

ENV PYTHONUNBUFFERED=1
# PYTHONUNBUFFERED is important specifically because of how we log: stdout
# must be flushed line-by-line for Cloud Logging to pick up each JSON log
# entry promptly rather than buffered in chunks.

# Cloud Run injects the actual port to listen on via the $PORT env var at
# runtime – it is NOT guaranteed to be 8080, so we never hardcode it. The
# EXPOSE here is documentation only; the real bind happens in CMD below.
EXPOSE 8080

CMD ["sh", "-c", "uvicorn server:app --host 0.0.0.0 --port ${PORT:-8080}"]

```

3.2 mcp_server/server.py

```

"""
MCP server entrypoint.

Wires the official MCP Python SDK (FastMCP) into a FastAPI ASGI app over
the streamable-HTTP transport, with three middleware layered in front of
the MCP tool dispatch in the order: auth -> rate limit -> logging.

Run locally with:
    uvicorn server:app --host 0.0.0.0 --port 8080 --reload
"""

import contextlib
import os
from datetime import datetime, timezone

from fastapi import FastAPI
from fastapi.responses import JSONResponse
from mcp.server.fastmcp import FastMCP

from middleware.auth import APIKeyAuthMiddleware
from middleware.rate_limit import SlidingWindowRateLimitMiddleware
from middleware.logging import (
    ToolCallLoggingMiddleware,
    configure_logging,
    log_tool_invocation,
)

configure_logging()

```

```
# stateless_http=True: each HTTP request is handled independently with no
# server-side session affinity required between requests. This matters on
# Cloud Run, where there is no guarantee two requests from the same client
# land on the same instance.
mcp = FastMCP("enterprise-mcp-server", stateless_http=True)
```

```
@mcp.tool()
@log_tool_invocation
def get_server_time() -> dict:
    """Return the current UTC server time. Useful as a connectivity /
    health-check tool."""
    now = datetime.now(timezone.utc)
    return {"utc_time": now.isoformat()}
```

```
@mcp.tool()
@log_tool_invocation
def echo_message(message: str, uppercase: bool = False) -> dict:
    """Echo a message back to the caller, optionally uppercased."""
    if uppercase:
        message = message.upper()
    return {"echo": message}
```

```
# The FastMCP-provided ASGI app for the streamable-HTTP transport. By
# default this app serves the MCP protocol endpoint at the path "/mcp" –
# mounting it at "/" below means the final externally-reachable path is
# exactly "/mcp" (a POST to "/mcp" without the trailing slash will receive
# a 307 redirect to "/mcp/"; MCP clients follow redirects automatically,
# but be aware of this if you're testing with curl -- see Section 5).
mcp_asgi_app = mcp.streamable_http_app()
```

```
@contextlib.asynccontextmanager
async def lifespan(app: FastAPI):
    # FastMCP's streamable-HTTP transport needs its session manager's
    # task group running for the lifetime of the app -- without this,
    # every request raises "Task group is not initialized." This is the
    # single most common setup mistake when mounting FastMCP into an
    # existing FastAPI app, so don't skip it.
    async with mcp.session_manager.run():
        yield
```

```
app = FastAPI(title="Enterprise MCP Server", lifespan=lifespan)
```

```
@app.get("/healthz")
async def healthz():
    return JSONResponse({"status": "ok"})
```

```
# Middleware order matters and is the inverse of the order you might
```

```
# expect from reading top-to-bottom: Starlette/FastAPI wraps middleware
# such that the LAST one added is the OUTERMOST layer, i.e. it runs FIRST
# on an incoming request. To get the execution order
#   auth -> rate limit -> logging -> tool dispatch
# we must add them in exactly this order: auth first, rate limit second,
# logging last. See Section 4 for the full reasoning and a worked trace
# of how Starlette builds this stack.
app.add_middleware(APIKeyAuthMiddleware)
app.add_middleware(SlidingWindowRateLimitMiddleware)
app.add_middleware(ToolCallLoggingMiddleware)

# Mounted last so the /healthz route (registered above) and any other
# direct routes on `app` are matched before the catch-all MCP mount.
app.mount("/", mcp_asgi_app)

if __name__ == "__main__":
    import uvicorn

    port = int(os.environ.get("PORT", "8080"))
    uvicorn.run(app, host="0.0.0.0", port=port)
```

3.3 mcp_server/middleware/__init__.py

```
"""Middleware package for the MCP server: auth, rate limiting, and
logging."""
```

3.4 mcp_server/middleware/auth.py

```
"""
API-key authentication middleware.

Fail-fast policy: if VALID_API_KEYS is missing or malformed, this module
raises at IMPORT time, which means the container process exits non-zero
on startup, before it ever binds to $PORT or accepts a single request.
We deliberately do not fall back to "no auth" or "allow everything" --
a server that refuses to start is a far smaller incident than a server
that quietly starts up with authentication disabled.
"""

import os

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse

# Paths that are reachable without an API key. Kept intentionally tiny --
# only the Cloud Run health check needs this exemption.
_PUBLIC_PATHS = frozenset({" /healthz "})
```



```

def _parse_valid_api_keys(raw: str | None) -> dict[str, str]:
    if not raw or not raw.strip():
        raise RuntimeError(
            "FATAL: VALID_API_KEYS environment variable is not set. "
            "Expected a comma-separated list of 'key:client_name' pairs, "
            "e.g. 'sk_live_abc123:acme-corp,sk_live_xyz789:globex-inc'. "
            "Refusing to start with no valid keys configured."
        )

    keys: dict[str, str] = {}
    for raw_pair in raw.split(","):
        pair = raw_pair.strip()
        if not pair:
            continue
        if ":" not in pair:
            raise RuntimeError(
                f"FATAL: malformed entry in VALID_API_KEYS: '{pair}'. "
                "Expected format 'key:client_name'."
            )
        key, _, client_name = pair.partition(":")
        key = key.strip()
        client_name = client_name.strip()
        if not key or not client_name:
            raise RuntimeError(
                f"FATAL: malformed entry in VALID_API_KEYS: '{pair}'. "
                "Both key and client_name must be non-empty."
            )
        keys[key] = client_name

    if not keys:
        raise RuntimeError("FATAL: VALID_API_KEYS parsed to zero usable keys.")

    return keys

```

```

# Parsed once, at import time. If this raises, the container never starts.
VALID_API_KEYS: dict[str, str] =
    _parse_valid_api_keys(os.environ.get("VALID_API_KEYS"))

```

```

class APIKeyAuthMiddleware(BaseHTTPMiddleware):
    """
    Rejects any request that does not carry a valid `x-api-key` header.

    Runs BEFORE rate limiting and logging (see server.py for the
    add_middleware
    order and Section 4 for why), so that unauthenticated traffic never
    consumes a legitimate client's rate-limit budget and is never logged
    as a normal tool call.
    """

```

```

async def dispatch(self, request: Request, call_next):
    if request.url.path in _PUBLIC_PATHS:
        return await call_next(request)

    api_key = request.headers.get("x-api-key")

    if not api_key:
        return JSONResponse(
            status_code=401,
            content={
                "error": "unauthorized",
                "message": "Missing required header: x-api-key",
            },
        )

    client_name = VALID_API_KEYS.get(api_key)
    if client_name is None:
        return JSONResponse(
            status_code=401,
            content={"error": "unauthorized", "message": "Invalid x-
api-key"},
        )

    # Stashed on request.state so downstream middleware (rate limiter,
    # logger) know who the caller is without re-validating the key.
    request.state.api_key = api_key
    request.state.client_name = client_name

    return await call_next(request)

```

3.5 mcp_server/middleware/rate_limit.py

```

"""
In-memory sliding-window rate limiter: max 60 requests / 60 seconds per
API key.

```

LIMITATION (documented per the project's stated assumptions): the `\_request\_log` dict below lives in the memory of a single container process. If Cloud Run scales this service to more than one instance, each instance maintains its own independent counter for a given API key -- there is no shared state between instances. A client could then send up to $(60 * N)$ requests/minute across N instances before any single instance throttles them, which silently breaks the "60 req/min per key" guarantee advertised by this service.

This is why `deploy.sh` pins `--min-instances=1 --max-instances=1`: with exactly one instance, this in-memory counter is the only counter, so it is correct by construction. It is a workable shortcut for this exercise and a hard outage risk in real production traffic (a single pinned instance means no autoscaling and no failover).

The real production fix is to move the counter into a shared, external store -- Redis or Google Cloud Memorystore for Redis -- keyed the same way (per API key, sliding window), so every Cloud Run instance reads and writes the same counters and the limit holds regardless of instance count. That swap touches only this file: the `dispatch()` logic below would become a Redis `ZADD/ZREMRANGEBYSCORE/ZCARD` sequence (or a Lua script for atomicity) instead of an in-process deque, while the middleware's external behavior (429 + Retry-After) stays identical.

 """"

```
import threading
import time
from collections import defaultdict, deque
```

```
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse
```

```
WINDOW_SECONDS = 60
MAX_REQUESTS_PER_WINDOW = 60
```

```
_PUBLIC_PATHS = frozenset({"/healthz"})
```

```
_lock = threading.Lock()
_request_log: dict[str, deque] = defaultdict(deque)
```

```
class SlidingWindowRateLimitMiddleware(BaseHTTPMiddleware):
    """
```

```
    Runs AFTER auth (so we already know which API key to bucket the
    request under) and BEFORE logging (so a throttled request is logged
    with status 429, rather than never reaching the logger at all).
    """
```

```
    async def dispatch(self, request: Request, call_next):
        if request.url.path in _PUBLIC_PATHS:
            return await call_next(request)
```

```
        api_key = getattr(request.state, "api_key", None)
        if api_key is None:
            # Unreachable in normal operation: auth middleware runs first
            # and would already have rejected an unauthenticated request.
            # Fail safe anyway rather than rate-limiting an unknown caller.
            return JSONResponse(status_code=401, content={"error":
"unauthorized"})
```

```
        now = time.monotonic()
```

```
        with _lock:
            bucket = _request_log[api_key]
```

```
            # Evict timestamps that have aged out of the trailing window.
```

```

        while bucket and (now - bucket[0]) > WINDOW_SECONDS:
            bucket.popleft()

        if len(bucket) >= MAX_REQUESTS_PER_WINDOW:
            oldest = bucket[0]
            seconds_until_oldest_expires = WINDOW_SECONDS - (now -
oldest)

            retry_after = max(1, int(seconds_until_oldest_expires) + 1)
            return JsonResponse(
                status_code=429,
                content={
                    "error": "rate_limited",
                    "message": (
                        f"Rate limit exceeded:
{MAX_REQUESTS_PER_WINDOW} "
                        f"requests per {WINDOW_SECONDS}s per API key."
                    ),
                },
                headers={"Retry-After": str(retry_after)},
            )

        bucket.append(now)

    return await call_next(request)

```

3.6 mcp_server/middleware/logging.py

```

"""
Dual logging: every tool call logs to stdout as structured JSON (so Cloud
Run / Cloud Logging picks it up automatically) AND to a local output.log
file in the plain-text format `TIMESTAMP | LEVEL | MESSAGE`.

Two complementary pieces of instrumentation live here:

1. `ToolCallLoggingMiddleware` -- an HTTP-transport-level middleware that
wraps every request (including ones rejected by auth or throttled by
the rate limiter), extracts Cloud Run's automatically-injected
`X-Cloud-Trace-Context` header, and logs a transport-level summary
line. It stores the trace id in a ContextVar for the lifetime of the
request.

2. `log_tool_invocation` -- a decorator applied directly to each
`@mcp.tool()` function. Because it wraps the actual Python function
call rather than the HTTP request, it has direct access to the real
tool name and the real arguments the tool was called with -- exactly
what the project requires ("tool name, args, duration, status") --
without needing to parse the MCP JSON-RPC body out of the raw ASGI
request stream (which is fragile to do correctly inside HTTP
middleware without breaking the streamable-HTTP transport's own body
handling). It reads the same trace id out of the ContextVar that (1)
set, so a tool-call log line and the HTTP request it happened inside

```

```

    carry the same trace id and can be correlated in Cloud Console.
    """

import contextvars
import functools
import json
import logging
import os
import time
import uuid

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request

LOG_FILE_PATH = os.environ.get("LOCAL_LOG_FILE", "output.log")
LOGGER_NAME = "mcp_server"

# Shared between the HTTP-level middleware (which extracts the trace id
# from Cloud Run's X-Cloud-Trace-Context header) and the tool-call
# decorator (which has no direct handle on the HTTP request object but
# still needs the same trace id so its log line correlates with the
# matching Cloud Trace span).
current_trace_id: contextvars.ContextVar[str] = contextvars.ContextVar(
    "current_trace_id", default="no-trace"
)

class _JsonStdoutFormatter(logging.Formatter):
    """Formats each log record as a single line of JSON on stdout, which
    Cloud Run's logging agent parses as one structured LogEntry per
    line."""

    def format(self, record: logging.LogRecord) -> str:
        payload = {
            "severity": record.levelname,
            "message": record.getMessage(),
            "logger": record.name,
            "timestamp": self.formatTime(record, "%Y-%m-%dT%H:%M:%S%z"),
        }
        structured_fields = getattr(record, "structured_fields", None)
        if structured_fields:
            payload.update(structured_fields)
        return json.dumps(payload)

def configure_logging() -> None:
    """
    Attaches a StreamHandler (JSON, stdout -> Cloud Logging on Cloud Run)
    and a FileHandler (plain text, output.log) to the `mcp_server`
    logger. Safe to call more than once -- it's a no-op if handlers are
    already attached, so re-importing this module doesn't double-log.
    """
    logger = logging.getLogger(LOGGER_NAME)
    logger.setLevel(logging.INFO)

```

```

logger.propagate = False

if logger.handlers:
    return

stream_handler = logging.StreamHandler()
stream_handler.setFormatter(_JsonStdoutFormatter())
logger.addHandler(stream_handler)

file_handler = logging.FileHandler(LOG_FILE_PATH)
file_handler.setFormatter(
    logging.Formatter(fmt="%(asctime)s | %(levelname)s | %(message)s")
)
logger.addHandler(file_handler)

logger = logging.getLogger(LOGGER_NAME)

_PUBLIC_PATHS = frozenset({"healthz"})

class ToolCallLoggingMiddleware(BaseHTTPMiddleware):
    """
    HTTP-transport-level access log. This is the innermost of the three
    middleware (added last in server.py, so it runs last on the way in --
    right before the MCP tool dispatch -- and first on the way out).
    By the time a request reaches here it has already cleared auth and
    rate-limiting, OR it was rejected by one of them; either way we log
    the outcome with the real status code.
    """

    async def dispatch(self, request: Request, call_next):
        start = time.perf_counter()

        trace_header = request.headers.get("x-cloud-trace-context", "")
        # Cloud Run / the GFE inject this header as "TRACE_ID/SPAN_ID;o=1".
        # Only the portion before the "/" is the trace id Cloud Trace
        # groups spans under.
        trace_id = trace_header.split("/")[0] if trace_header else
str(uuid.uuid4())
        token = current_trace_id.set(trace_id)

        client_name = getattr(request.state, "client_name", "unknown")

        try:
            response = await call_next(request)
        finally:
            current_trace_id.reset(token)

        duration_ms = round((time.perf_counter() - start) * 1000, 2)

        if request.url.path not in _PUBLIC_PATHS:
            logger.info(
                f"http_request path={request.url.path} client={client_name}

```

```

        f"status={response.status_code} duration_ms={duration_ms}
trace_id={trace_id}",
        extra={
            "structured_fields": {
                "event": "http_request",
                "path": request.url.path,
                "client": client_name,
                "status": response.status_code,
                "duration_ms": duration_ms,
                "trace_id": trace_id,
            }
        },
    )

    # Surfaced back to the caller so a curl client (or trace_ids.md)
    # can read the trace id straight out of the response headers
    # without having to separately know about X-Cloud-Trace-Context.
    response.headers["X-Trace-Id"] = trace_id
    return response

```

```
def log_tool_invocation(func):
```

```
    """
```

Decorator for `@mcp.tool()` functions. Logs exactly the fields the project requires per call: tool name, arguments, duration in ms, and status -- tagged with the trace id captured by ToolCallLoggingMiddleware for the HTTP request this call happened inside of.

Works for both sync and async tool functions. Apply it BELOW @mcp.tool() (i.e. closer to the function definition) so that FastMCP registers the already-instrumented callable:

```

    @mcp.tool()
    @log_tool_invocation
    def my_tool(...): ...
    """

```

```
@functools.wraps(func)
```

```
async def wrapper(*args, **kwargs):
```

```
    start = time.perf_counter()
```

```
    status = "success"
```

```
    try:
```

```
        result = func(*args, **kwargs)
```

```
        if hasattr(result, "__await__"):
```

```
            result = await result
```

```
        return result
```

```
    except Exception:
```

```
        status = "error"
```

```
        raise
```

```
    finally:
```

```
        duration_ms = round((time.perf_counter() - start) * 1000, 2)
```

```
        trace_id = current_trace_id.get()
```

```

        named_args = dict(kwargs)
        if args:
            named_args["_positional_args"] = list(args)
        logger.info(
            f"tool_call tool={func.__name__} status={status} "
            f"duration_ms={duration_ms} trace_id={trace_id}",
            extra={
                "structured_fields": {
                    "event": "tool_call",
                    "tool_name": func.__name__,
                    "args": named_args,
                    "duration_ms": duration_ms,
                    "status": status,
                    "trace_id": trace_id,
                }
            },
        )

    return wrapper

```

3.7 mcp_server/deploy.sh

```

#!/usr/bin/env bash
set -euo pipefail

# -----
--
# deploy.sh -- builds the container image, pushes it to Artifact Registry,
# and deploys it to Cloud Run.
#
# REQUIRED before running:
#   export PROJECT_ID=YOUR_PROJECT_ID
#   export VALID_API_KEYS="sk_live_xxx:client_a,sk_live_yyy:client_b"
#
# Optional overrides (defaults shown):
#   export REGION=us-central1
#   export REPO_NAME=mcp-server-repo
#   export SERVICE_NAME=mcp-server
# -----
--

PROJECT_ID="${PROJECT_ID:-YOUR_PROJECT_ID}"
REGION="${REGION:-us-central1}"
REPO_NAME="${REPO_NAME:-mcp-server-repo}"
SERVICE_NAME="${SERVICE_NAME:-mcp-server}"
IMAGE_NAME="${REGION}-
docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/${SERVICE_NAME}"
IMAGE_TAG="${IMAGE_TAG:-$(date +%Y%m%d-%H%M%S)}"

if [[ "${PROJECT_ID}" == "YOUR_PROJECT_ID" ]]; then
    echo "ERROR: set PROJECT_ID (export PROJECT_ID=...) before deploying."

```



```

>&2
    exit 1
fi

if [[ -z "${VALID_API_KEYS:-}" ]]; then
    echo "ERROR: VALID_API_KEYS must be set before deploying, e.g.:" >&2
    echo "    export
VALID_API_KEYS='sk_live_xxx:client_a,sk_live_yyy:client_b'" >&2
    exit 1
fi

echo "==> Project: ${PROJECT_ID}   Region: ${REGION}   Service:
${SERVICE_NAME}"

echo "==> Ensuring Artifact Registry repo '${REPO_NAME}' exists in
${REGION}..."
if ! gcloud artifacts repositories describe "${REPO_NAME}" \
    --location="${REGION}" \
    --project="${PROJECT_ID}" >/dev/null 2>&1; then
    gcloud artifacts repositories create "${REPO_NAME}" \
        --repository-format=docker \
        --location="${REGION}" \
        --project="${PROJECT_ID}" \
        --description="MCP server container images"
fi

echo "==> Configuring Docker auth for Artifact Registry..."
gcloud auth configure-docker "${REGION}-docker.pkg.dev" --quiet

echo "==> Building image ${IMAGE_NAME}:${IMAGE_TAG}..."
docker build --platform linux/amd64 \
    -t "${IMAGE_NAME}:${IMAGE_TAG}" \
    -t "${IMAGE_NAME}:latest" \
    .

echo "==> Pushing image..."
docker push "${IMAGE_NAME}:${IMAGE_TAG}"
docker push "${IMAGE_NAME}:latest"

echo "==> Deploying to Cloud Run..."
gcloud run deploy "${SERVICE_NAME}" \
    --project="${PROJECT_ID}" \
    --image="${IMAGE_NAME}:${IMAGE_TAG}" \
    --region="${REGION}" \
    --platform=managed \
    --memory=512Mi \
    --cpu=1 \
    --min-instances=1 \
    --max-instances=1 \
    --concurrency=80 \
    --timeout=300 \
    --port=8080 \
    --allow-unauthenticated \
    --set-env-vars="VALID_API_KEYS=${VALID_API_KEYS}"

```

```
# --allow-unauthenticated: IAM-level auth is intentionally OFF. Auth for
# this service is enforced at the application layer (x-api-key, see
# middleware/auth.py), not at the Cloud Run/IAM layer. If you also wanted
# IAM-level protection (e.g. only allow calls from a specific service
# account), you'd drop this flag and grant roles/run.invoker explicitly --
# but that's a different access model than "any client with a valid API
# key," which is what this exercise calls for.
#
# --min-instances=1 --max-instances=1: pins the service to exactly one
# instance so the in-memory sliding-window rate limiter (rate_limit.py)
# has a single, authoritative counter. See the limitation documented in
# that file and in Section 1.1 / Section 4.

echo "==> Fetching service URL..."
SERVICE_URL=$(gcloud run services describe "${SERVICE_NAME}" \
  --project="${PROJECT_ID}" \
  --region="${REGION}" \
  --format="value(status.url)")

OUTPUT_FILE="../../mcp_deploy/cloud_run_url.txt"
if [[ -d "../../mcp_deploy" ]]; then
  echo "${SERVICE_URL}" > "${OUTPUT_FILE}"
  echo "==> URL written to ${OUTPUT_FILE}"
fi

echo "==> Deployed. Live URL: ${SERVICE_URL}"
```

3.8 mcp_server/requirements.txt

```
fastapi>=0.115,<0.116
uvicorn[standard]>=0.32,<0.33
starlette>=0.40,<0.41
mcp>=1.9,<2.0
```

3.9 mcp_server/.env.example

```
# Comma-separated key:client_name pairs. REPLACE before running locally
# or deploying -- these example values are not real keys.
# Production note: see mcp_deploy/auth_setup.md for the Secret-Manager-
# backed
# alternative to plaintext env vars.
VALID_API_KEYS=sk_local_REPLACE_ME:local-dev,sk_local_REPLACE_ME_2:qa-team

# Local-only. Cloud Run injects PORT automatically at deploy time; this is
# only read when running `uvicorn server:app` or `python server.py` on
# your own machine.
PORT=8080
```

```
# Optional override for the local structured-log file path written by
# middleware/logging.py. Defaults to ./output.log if unset.
LOCAL_LOG_FILE=output.log
```

3.10 mcp_server/.dockerignore

```
myenv/
venv/
__pycache__/
*.pyc
*.pyo
.env
.env.example
output.log
.git/
.gitignore
.dockerignore
Dockerfile
deploy.sh
*.md
```

4. Code Logic & Deep-Dive

Why the multi-stage Dockerfile actually reduces image size. The **builder** stage starts from **python:3.12-slim** and runs **pip install**, which for some dependencies (cryptography-adjacent packages, anything with C extensions) needs build tooling that doesn't ship in the slim base and would otherwise have to be **apt-get install**'d. Whatever ends up in that builder stage's filesystem — apt package lists, pip's wheel cache, any temporary build artifacts — never gets copied forward. The **runtime** stage starts fresh from **python:3.12-slim** again and copies in exactly one thing from the builder: **/opt/venv**, the already-built virtual environment, plus our own **server.py** and **middleware/**. Docker layers are content-addressed and cached independently per stage, so this also means re-running **docker build** after only touching **server.py** doesn't reinstall dependencies — the **builder** stage's layers are unchanged and Docker reuses its cache.

Why middleware order is auth → rate-limit → logging, and how Starlette actually builds that order. This trips people up because the code reads in the order you'd execute it, but Starlette's **add_middleware** builds the stack by wrapping outward — the *last* middleware you add becomes the *outermost* layer, meaning it's the *first* one to see an incoming request. Concretely, in **server.py** we call:

```
app.add_middleware(APIKeyAuthMiddleware)
app.add_middleware(SlidingWindowRateLimitMiddleware)
app.add_middleware(ToolCallLoggingMiddleware)
```

Starlette stores these in **user_middleware** in call order [**Auth**, **RateLimit**, **Logging**], then builds the actual ASGI call chain by iterating that list *in reverse* and wrapping each one around the previous

result: `Logging` wraps the bare app first, `RateLimit` wraps that, and `Auth` wraps everything built so far. The result is `Auth(RateLimit(Logging(app)))` — so on an incoming request, `Auth` runs first, then `RateLimit`, then `Logging`, then the actual MCP dispatch. This is exactly the order we want, for a simple reason: each layer should only do its job for requests that deserve to reach it. An unauthenticated request shouldn't burn a slot in a legitimate client's rate-limit budget (so auth must run before rate-limiting), and a request that gets rejected by either of those two should still be logged as a 401 or 429 with accurate timing (so logging has to wrap the innermost layer, closest to the real work, so its `duration_ms` reflects the full cost of whatever happened — including time spent in the layers above it, since the duration timer in `ToolCallLoggingMiddleware` starts before `call_next` and `call_next` is what invokes everything downstream of it, i.e. the actual tool dispatch only).

How the sliding-window counter is computed and evicted. `rate_limit.py` keeps one deque of timestamps per API key, all using `time.monotonic()` (never `time.time()`, which can jump backward on clock corrections — a monotonic clock can't). On every request: pop timestamps off the left of the deque as long as they're older than `WINDOW_SECONDS` (60s) relative to *now* — this is the eviction step, and it happens lazily on each request rather than via a background sweep, which keeps the implementation simple at the cost of slightly stale deques for keys that go quiet (an idle key's deque just sits there until its next request triggers eviction, but it's bounded at 60 entries max so this is never a memory concern). If, after eviction, the deque's length is still `>= 60`, the request is over budget and gets a 429 with a computed `Retry-After` — specifically, the number of seconds until the *oldest* timestamp in the window ages out, which is the earliest moment the request could succeed if retried. If under budget, the current timestamp is appended and the request proceeds.

How the trace ID is captured and threaded through so logs and traces correlate in Cloud Console.

Cloud Run (specifically the Google Front End in front of it) automatically attaches an `X-Cloud-Trace-Context` header to every inbound request, formatted as `TRACE_ID/SPAN_ID;o=TRACE_TRUE`.

`ToolCallLoggingMiddleware` reads this header, takes the portion before the `/` (that's the trace ID Cloud Trace groups spans under), and stores it in a `contextvars.ContextVar` for the duration of that single request — ContextVars are the correct primitive here rather than a plain module-level variable, because Cloud Run can be serving several concurrent requests inside one container instance, and a plain variable would get clobbered by whichever request wrote to it last; a ContextVar is automatically scoped per `asyncio` task, so concurrent requests each see their own value. The `log_tool_invocation` decorator on each tool function reads that same ContextVar when it logs, so its structured log entry carries the identical `trace_id` field as the HTTP-level log line for the request it ran inside of. In Cloud Console, you'd search Cloud Logging by `jsonPayload.trace_id="<value>"` (or use Cloud Trace's own log-correlation panel, which Cloud Run wires up automatically when it recognizes the trace context format) and see both the transport-level `http_request` entry and the tool-level `tool_call` entry side by side, plus the actual trace/span timeline in Cloud Trace.

Why the server reads `PORT` from the environment instead of hardcoding it. Cloud Run injects a `PORT` environment variable into the container at startup and expects the container's process to bind to it — this value is an implementation detail of the platform and is not guaranteed to be `8080` (it has been historically, but the contract is "whatever Cloud Run tells you," not "always 8080"). Hardcoding a port means the container silently fails to receive traffic if that assumption ever breaks, with no error message pointing at the actual cause. Both `server.py` (`int(os.environ.get("PORT", "8080"))`), used for local fallback) and the Dockerfile's `CMD` (`--port ${PORT:-8080}`), a shell-level default for the same

reason) read it dynamically, with `8080` only ever used as a local-development fallback when `PORT` isn't set at all.

5. Deployment & Execution Guide

All commands below assume macOS with `zsh` as the shell, and that you're running them from inside `mcp_server/` unless otherwise noted.

5.1 Install and authenticate the `gcloud` CLI

```
# If gcloud isn't installed yet:
brew install --cask google-cloud-sdk

gcloud init
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```

5.2 Create and activate `myenv`, install dependencies

```
cd mcp_server
python3 -m venv myenv
source myenv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

5.3 Local test run

```
cp .env.example .env
# Edit .env and replace the placeholder VALID_API_KEYS values with real
# local test keys before continuing.

export $(grep -v '^#' .env | xargs)
uvicorn server:app --host 0.0.0.0 --port 8080 --reload
```

In a second terminal, with the same key you put in `.env`:

```
curl -s -i \
  -X POST "http://localhost:8080/mcp/" \
  -H "x-api-key: sk_local_REPLACE_ME" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":
{"name":"get_server_time","arguments":{}}}'
```

You should see a **200** response and, in the terminal running **uvicorn**, both a JSON **http_request** log line and a JSON **tool_call** log line on stdout, plus matching lines appended to **output.log** in **TIMESTAMP | LEVEL | MESSAGE** format. Try it again without **-H "x-api-key: ..."** and confirm you get a **401**.

5.4 Build and push the image to Artifact Registry

deploy.sh does this for you (next step), but to do it by hand first as a sanity check:

```
export PROJECT_ID=YOUR_PROJECT_ID
export REGION=us-central1
export REPO_NAME=mcp-server-repo

gcloud artifacts repositories create "${REPO_NAME}" \
  --repository-format=docker \
  --location="${REGION}" \
  --description="MCP server container images" \
  || echo "Repo already exists, continuing."

gcloud auth configure-docker "${REGION}-docker.pkg.dev" --quiet

docker build --platform linux/amd64 \
  -t "${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/mcp-
server:manual-test" \
  .

docker push "${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/mcp-
server:manual-test"
```

5.5 Run **deploy.sh** and capture the live URL

```
export PROJECT_ID=YOUR_PROJECT_ID
export VALID_API_KEYS="sk_live_REPLACE_ME:acme-
corp,sk_live_REPLACE_ME_2:globex-inc"

chmod +x deploy.sh
./deploy.sh
```

deploy.sh writes the resulting Cloud Run URL into **../mcp_deploy/cloud_run_url.txt** automatically. Verify:

```
cat ../mcp_deploy/cloud_run_url.txt
```

5.6 Five example calls against the live URL, capturing trace IDs

```
SERVICE_URL=$(cat ../mcp_deploy/cloud_run_url.txt)
API_KEY="sk_live_REPLACE_ME"

curl -s -i \
  -X POST "${SERVICE_URL}/mcp/" \
  -H "x-api-key: ${API_KEY}" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":
{"name":"get_server_time","arguments":{}}}'
```

Run that five times (varying the `"id"` and, for a couple of calls, switching to `echo_message` with `"arguments":{"message":"hello","uppercase":true}`). For each call, the response headers from `curl -i` include a line like:

```
x-trace-id: 6b1f3c2a9e8d7f4a3b2c1d0e9f8a7b6c
```

That value is exactly what goes in the **Trace ID** column of `mcp_deploy/trace_ids.md` for that call, alongside the timestamp, the tool called, the input arguments, and the output you got back. This guide can show you the command pattern, but the actual five rows in `trace_ids.md` — and the Cloud Trace / Cloud Logging screenshots — have to come from values you get back from a real deployed service; they can't be fabricated here.

5.7 Pull matching entries from Cloud Logging, open Cloud Trace

```
gcloud logging read \
  'resource.type="cloud_run_revision" AND
  resource.labels.service_name="mcp-server" AND
  jsonPayload.event="tool_call"' \
  --project="${PROJECT_ID}" \
  --limit=20 \
  --format=json
```

Filter to one specific call by trace ID once you have it:

```
gcloud logging read \
  "resource.type=\"cloud_run_revision\" AND
  resource.labels.service_name=\"mcp-server\" AND
  jsonPayload.trace_id=\"PASTE_TRACE_ID_HERE\"" \
  --project="${PROJECT_ID}" \
  --format=json
```

Open Cloud Trace in the console to find and screenshot the matching trace:

```
open "https://console.cloud.google.com/traces/list?project=${PROJECT_ID}"
```

And Cloud Logging directly:

```
open "https://console.cloud.google.com/logs/query?project=${PROJECT_ID}"
```

5.8 `setup.sh` (separate from `deploy.sh`)

A standalone scaffolding script, distinct from `deploy.sh`. It performs the same project scaffolding as Section 2's script, and additionally creates a blank `README.md` containing only an H1 title placeholder — no body content, since that's left for you to fill in.

```
#!/usr/bin/env zsh
set -euo pipefail

# setup.sh - scaffolds the mcp_server / mcp_deploy project layout, creates
# the `myenv` virtual environment, and seeds a bare README.md with just an
# H1 placeholder. Idempotent: safe to re-run. Never overwrites an existing
# README.md that already has content in it.

ROOT="$(pwd)"

echo "==> Creating directory structure under ${ROOT}"
mkdir -p "${ROOT}/mcp_server/middleware"
mkdir -p "${ROOT}/mcp_deploy"

echo "==> Creating mcp_server/ files"
touch "${ROOT}/mcp_server/server.py"
touch "${ROOT}/mcp_server/requirements.txt"
touch "${ROOT}/mcp_server/Dockerfile"
touch "${ROOT}/mcp_server/deploy.sh"
chmod +x "${ROOT}/mcp_server/deploy.sh"
touch "${ROOT}/mcp_server/.env.example"
touch "${ROOT}/mcp_server/.dockerignore"

echo "==> Creating mcp_server/middleware/ files"
touch "${ROOT}/mcp_server/middleware/__init__.py"
touch "${ROOT}/mcp_server/middleware/auth.py"
touch "${ROOT}/mcp_server/middleware/rate_limit.py"
touch "${ROOT}/mcp_server/middleware/logging.py"

echo "==> Creating mcp_deploy/ files"
touch "${ROOT}/mcp_deploy/cloud_run_url.txt"
touch "${ROOT}/mcp_deploy/trace_ids.md"
touch "${ROOT}/mcp_deploy/auth_setup.md"

echo "==> Creating top-level files"
touch "${ROOT}/questions.md"
```



```
touch "${ROOT}/output.log"

README_PATH="${ROOT}/README.md"
if [[ ! -s "${README_PATH}" ]]; then
    printf '# Project Title\n' > "${README_PATH}"
    echo "==> Wrote blank H1 placeholder to README.md"
else
    echo "==> README.md already has content, leaving it untouched"
fi

echo "==> Setting up Python virtual environment (myenv)"
if [[ ! -d "${ROOT}/myenv" ]]; then
    python3 -m venv myenv
    echo "    created ./myenv"
else
    echo "    ./myenv already exists, skipping"
fi

echo "==> Done. Structure created/verified under ${ROOT}"
echo "    Activate the venv with: source myenv/bin/activate"
```

6. Intern Viva & Code Review Questions (questions.md Format)

Project Evaluation & Code Review

Q1: What is the purpose of a multi-stage Dockerfile, and which artifacts from the builder stage actually make it into the final runtime image in this project?

****Answer:****

Write your answer here...

Q2: In `server.py`, `app.add\_middleware()` is called in the order Auth, RateLimit, Logging – but the request execution order is also Auth, RateLimit, Logging. Explain why that's not a coincidence, and what would happen to a request that fails auth if the call order were reversed.

****Answer:****

Write your answer here...

Q3: Why does `middleware/auth.py` raise an exception at module import time rather than returning a 500 error the first time a request comes in with `VALID\_API\_KEYS` unset?

****Answer:****

Write your answer here...

Q4: Walk through exactly what happens, line by line, in `SlidingWindowRateLimitMiddleware.dispatch()` when a 61st request arrives from the same API key within a 60-second window. What does the returned `Retry-After` value actually represent?

****Answer:****

Write your answer here...

Q5: The Dockerfile reads `CMD ["sh", "-c", "uvicorn server:app --host 0.0.0.0 --port ${PORT:-8080}"]` instead of `CMD ["uvicorn", "server:app", "--port", "8080"]`. What would break on Cloud Run with the second form, and why does it matter that it's run through `sh -c` rather than passed as an exec-form argument list?

****Answer:****

Write your answer here...

Q6: Explain precisely why the in-memory sliding-window rate limiter in this project becomes incorrect the moment Cloud Run scales this service past one instance. What specific Cloud Run behavior (not a code bug) causes this?

****Answer:****

Write your answer here...

Q7: `deploy.sh` passes `--min-instances=1 --max-instances=1` to work around the issue in Q6. What are the real production costs of this workaround (think: availability, deploys, regional failure), and what would you replace it with for a multi-instance deployment serving real client traffic?

****Answer:****

Write your answer here...

Q8: How does `ToolCallLoggingMiddleware` correlate a structured log entry with a specific Cloud Trace span in Cloud Console? Be specific about which header is involved, who sets it, and why a `contextvars.ContextVar` is used instead of a plain module-level variable to pass the trace id from the middleware to `log_tool_invocation`.

****Answer:****

Write your answer here...

Q9: `middleware/auth.py` is strict/fail-fast: a missing or invalid `x-api-key` always returns 401 immediately, with no retry hint, no grace period, and no fallback to an "anonymous" tier. Argue both sides: when is this the right call for a service like this one, and when would a more graceful degradation strategy (e.g. a short-lived cache of last-known-good keys, or a soft warning period after a key rotation) be justified instead?

****Answer:****

Write your answer here...

Q10: Cloud Run can scale a service down to zero instances when idle. Describe, in order, everything that has to happen between an MCP client's first request after a cold scale-up and that request actually reaching `get_server_time()` – including image pull (or lack thereof), container start, the `lifespan` context manager in `server.py`, and the `VALID_API_KEYS` parsing in `auth.py` – and explain which of these steps actually contribute meaningfully to the cold-start latency a client would observe.

****Answer:****

Write your answer here...